Lab5A - Constructing a CNN Network

For spatial data for example image or video data, Convolutional Neural Network (CNN or ConvNet) performs much better than standard neural network. In this practical, we shall learn how to build a CNN Network.

Table 1: Result sheet for different configurations

Model	Best Accuracy
Standard Neural Network (kaiming_normal, relu, Batch Norm, 6 layers)	**42%**
CNN (2 conv and 2 fc, batchnorm, relu)	**58.2%**

Objectives:

1. Learn how to build a convolutional neural network.
2. Learn how to build a sequential model

Content:

1. Helper Functions
2. Load the CIFAR10 Dataset
3. Building a CNN Model using torch.nn.Module
4. Building a CNN Model using torch.nn.Sequential
5. Creating Block Layers

Commands:

1. Building a Module model
   `nn.Module` , `nn.Conv2d` , `nn.BatchNorm2d` , `nn.Linear1`
2. Displaying a model
   `torchsummary.summary`
3. Building a Sequential Model
   `nn.Sequential`

In [1]:

```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=947
318989803-6bn6qk8qdgf4n4g3pfee6491hc0brc4i.apps.googleusercontent.com&redirect_uri=u
rn%3aietf%3awg%3aoauth%3a2.0%3aoob&response_type=code&scope=email%20https%3a%2f%2fww
w.googleapis.com%2fauth%2fdocs.test%20https%3a%2f%2fwww.googleapis.com%2fauth%2fdriv
e%20https%3a%2f%2fwww.googleapis.com%2fauth%2fdrive.photos.readonly%20https%3a%2f%2f
www.googleapis.com%2fauth%2fpeopleapi.readonly

Enter your authorization code:
··********··
Mounted at /content/gdrive
```

In [2]:

```
cd "gdrive/My Drive/Lab5-Foundations_of_CNN"
```

```
/content/gdrive/My Drive/Lab5-Foundations_of_CNN
```

Import the required libraries

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms

import torch.optim as optim
from torch.utils.data import DataLoader

from torchsummary import summary

from cifar10 import CIFAR10

%load_ext autoreload
%autoreload 2
```

In [2]:
```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

1. Helper Functions

First we create two helper functions:

1. Use the `train` function to train the model on the training set. The train function will stop training if the average accuracy does not improve after three count.

In [3]:
```
def train(net, max_epochs, lr=0.1, momentum=0.9):

    # compute loss 3 times in each epoch
    loss_iterations = int(np.ceil(len(trainloader)/3))

    # transfer model to GPU
    net = net.to(device)

    # set the optimizer. Use SGD with momentum
    optimizer = optim.SGD(net.parameters(), lr=lr, momentum=momentum)

    # set to training mode
    net.train()

    # variables
    best_loss = np.inf
    saturate_count = 0

    # train the network
    for e in range(max_epochs):

        running_loss = 0
        running_count = 0

        for i, (inputs, labels) in enumerate(trainloader):

            # Clear all the gradient to 0
```

```
            optimizer.zero_grad()

            # transfer data to GPU
            inputs = inputs.to(device)
            labels = labels.to(device)

            # forward propagation to get h
            outs = net(inputs)

            # compute loss
            loss = F.cross_entropy(outs, labels)

            # backpropagation to get gradients of all parameters
            loss.backward()

            # update parameters
            optimizer.step()

            # get the loss
            running_loss += loss.item()
            running_count += 1

             # display the averaged loss value
            if i % loss_iterations == loss_iterations-1 or i == len(trainloader) - 1
                train_loss = running_loss / running_count
                running_loss = 0.
                running_count = 0.
                print(f'[Epoch {e+1:2d} Iter {i+1:5d}/{len(trainloader)}]: train_los

                if train_loss < best_loss:
                    best_loss = train_loss
                    saturate_count = 0
                else:
                    saturate_count += 1
                    if saturate_count >= 3:
                        return
```

1. Use the `evaluate` function to evaluate a trained model on the testing set.

In [4]:

```
def evaluate(model):

    # set to evaluation mode
    model.eval()

    # running_correct
    running_corrects = 0

    for inputs, targets in testloader:

        # transfer to the GPU
        inputs = inputs.to(device)
        targets = targets.to(device)

        # perform prediction (no need to compute gradient)
        with torch.no_grad():
            outputs = model(inputs)
            _, predicted = torch.max(outputs, 1)

            running_corrects += (targets == predicted).double().sum()


    print('Accuracy = {:.2f}%'.format(100*running_corrects/len(testloader.dataset)))
```

2. Load the CIFAR10 Dataset

The following code loads the dataset.

In [5]:

```
# transform the model
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

# dataset
trainset = CIFAR10(train=True,  transform=transform, num_samples=10000)
testset  = CIFAR10(train=False,  transform=transform, num_samples=2000)

# dataloader]
trainloader = DataLoader(trainset, batch_size=32, shuffle=True, num_workers=4)
testloader  = DataLoader(testset, batch_size=128, shuffle=True, num_workers=4)
```

---

3. Creating a CNN Model using torch.nn.Module

In this section, we create a CNN network using `nn.Module` . The `Module` is the main building block, it defines the base class for all neural network and you MUST subclass it.

Build the network

Let's create the following CNN. It is based on the AlexNet architechture which won the ImageNet competition in 2012 and triggered the deep learning revolution.

Layer	Name	Description	Input Shape	OutputShape
1	conv2d-1	Conv2d, f=3, c=32, s=1, p=1	(?, 3, 32, 32)	(?, 32, 32, 32)
2	BatchNorm2d-2	BatchNorm2d, c=32	(?, 32, 32, 32)	(?, 32, 32, 32)
-	-	relu	(?, 32, 32, 32)	(?, 32, 32, 32)
3	conv2d-3	Conv2d, f=3, c=64, s=1, p=0	(?, 32, 32, 32)	(?, 64, 30, 30)
4	BatchNorm2d-4	BatchNorm2d, c=64	(?, 64, 30, 30)	(?, 64, 30, 30)
-	Relu	relu	(?, 64, 30, 30)	(?, 64, 30, 30)
-	-	flatten	(?, 64, 30, 30)	(?, 57600)
5	Linear-5	Linear, c =1024	(?, 57600)	(?, 1024)
6	Linear-6	Linear, c =10	(?, 1024)	(?, 10)

Notes: **Batch normalization** is a layer that is added mainly to improve the performance of the system. It normalizes the activation signals across the batch data by subtracting with the batch mean and dividing with the batch standard deviation. Adding the layer enables training on deeper network, and also reduce overfitting. The layer always preserve the shape of the input volume.

Refer to Pytorch `torch.nn` for details on the following commands:

In [0]:

```
class Net(nn.Module):
```

```
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(32)

        self.conv2 = nn.Conv2d(32, 64, 3)
        self.bn2   = nn.BatchNorm2d(64)

        self.fc1 = nn.Linear(64*30*30, 1024)
        self.fc2 = nn.Linear(1024, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = F.relu(x)

        x = self.conv2(x)
        x = self.bn2(x)
        x = F.relu(x)

        x = x.view(x.size(0), -1) # flat
        x = self.fc1(x)
        x = self.fc2(x)

        return x
```

- Line 5: `nn.Conv2d(3, 32, 3, stride=1, padding=1)`
  - **First parameter (`in_c`)**:
    Number of input channels = 3. This must be the same as the number of channels for the previous layer. Since the previous layer is the input layer and each image has shape (3,32,32), the correct setting for `in_c` is 3.
  - **Second parameter (`out_c`)**:
    Number of output channels = 32. This is equivalent to the number of filters used in the layer.
  - **Third parameter: (`f`)**:
    Filter (or kernel) size = 3. This corresponds to the spatial dimension (height and width) of the filters. Hence, the spatial dimension of the filter is (2, 2)
  - Default settings (if not specified):
    stride=1
    padding=0,
    padding_mode='zeros'

You may refer to PyTorch manual on `torch.nn.Conv2d` for more details including how the weights are initialized.

Displaying the network

There are two ways to display the network.

1. Simply printing the network displays all the layers in the network. The order the layers are displayed are based on the sequence the layers are created in `__init__` , not the actual order they are called in `forward` . Hence, the order of the layers does not reflect the actual architecture of te network.

In [9]:

```
print(Net())
```

```
Net(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
  (fc1): Linear(in_features=57600, out_features=1024, bias=True)
  (fc2): Linear(in_features=1024, out_features=10, bias=True)
)
```

1. Using `torchsummary.summary` gives you a more accurate picture of the actual architecture since it is based on the order they are called in `forward`. It will display `nn.Module` layers (e.g., `torch.nn.ReLU`) and cannot functional layers (e.g., `torch.relu`).

In [10]:

```
summary(Net().to(device), input_size=(3, 32, 32), device=device)
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 32, 32, 32]             896
       BatchNorm2d-2           [-1, 32, 32, 32]              64
            Conv2d-3           [-1, 64, 30, 30]          18,496
       BatchNorm2d-4           [-1, 64, 30, 30]             128
            Linear-5                 [-1, 1024]      58,983,424
            Linear-6                   [-1, 10]          10,250
================================================================
Total params: 59,013,258
Trainable params: 59,013,258
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 1.39
Params size (MB): 225.12
Estimated Total Size (MB): 226.52
----------------------------------------------------------------
```

Train the model

In [11]:

```
model1 = Net()
train(model1, max_epochs=50, lr=0.001, momentum=0.9)
```

```
[Epoch  1 Iter   105/313]: train_loss = 1.9229
[Epoch  1 Iter   210/313]: train_loss = 1.5402
[Epoch  1 Iter   313/313]: train_loss = 1.4149
[Epoch  2 Iter   105/313]: train_loss = 1.0449
[Epoch  2 Iter   210/313]: train_loss = 1.0396
[Epoch  2 Iter   313/313]: train_loss = 1.0707
[Epoch  3 Iter   105/313]: train_loss = 0.7854
[Epoch  3 Iter   210/313]: train_loss = 0.8167
[Epoch  3 Iter   313/313]: train_loss = 0.8193
[Epoch  4 Iter   105/313]: train_loss = 0.5307
[Epoch  4 Iter   210/313]: train_loss = 0.6004
[Epoch  4 Iter   313/313]: train_loss = 0.6343
[Epoch  5 Iter   105/313]: train_loss = 0.3593
[Epoch  5 Iter   210/313]: train_loss = 0.3862
[Epoch  5 Iter   313/313]: train_loss = 0.4132
[Epoch  6 Iter   105/313]: train_loss = 0.2311
[Epoch  6 Iter   210/313]: train_loss = 0.2504
[Epoch  6 Iter   313/313]: train_loss = 0.2674
[Epoch  7 Iter   105/313]: train_loss = 0.1421
```

```
[Epoch  7 Iter   210/313]: train_loss = 0.1258
[Epoch  7 Iter   313/313]: train_loss = 0.1634
[Epoch  8 Iter   105/313]: train_loss = 0.0797
[Epoch  8 Iter   210/313]: train_loss = 0.0647
[Epoch  8 Iter   313/313]: train_loss = 0.0776
[Epoch  9 Iter   105/313]: train_loss = 0.0417
[Epoch  9 Iter   210/313]: train_loss = 0.0340
[Epoch  9 Iter   313/313]: train_loss = 0.0389
[Epoch 10 Iter   105/313]: train_loss = 0.0224
[Epoch 10 Iter   210/313]: train_loss = 0.0210
[Epoch 10 Iter   313/313]: train_loss = 0.0214
[Epoch 11 Iter   105/313]: train_loss = 0.0161
[Epoch 11 Iter   210/313]: train_loss = 0.0149
[Epoch 11 Iter   313/313]: train_loss = 0.0146
[Epoch 12 Iter   105/313]: train_loss = 0.0118
[Epoch 12 Iter   210/313]: train_loss = 0.0118
[Epoch 12 Iter   313/313]: train_loss = 0.0115
[Epoch 13 Iter   105/313]: train_loss = 0.0096
[Epoch 13 Iter   210/313]: train_loss = 0.0088
[Epoch 13 Iter   313/313]: train_loss = 0.0091
[Epoch 14 Iter   105/313]: train_loss = 0.0076
[Epoch 14 Iter   210/313]: train_loss = 0.0076
[Epoch 14 Iter   313/313]: train_loss = 0.0083
[Epoch 15 Iter   105/313]: train_loss = 0.0068
[Epoch 15 Iter   210/313]: train_loss = 0.0068
[Epoch 15 Iter   313/313]: train_loss = 0.0070
[Epoch 16 Iter   105/313]: train_loss = 0.0053
[Epoch 16 Iter   210/313]: train_loss = 0.0056
[Epoch 16 Iter   313/313]: train_loss = 0.0064
[Epoch 17 Iter   105/313]: train_loss = 0.0055
```

Evaluate the model

Now evaluate the model. Then, **update Table1**. Observe the difference in performance between standard neural network and CNN for this task.

In [12]:

```
evaluate(model1)
```

```
Accuracy = 58.50%
```

---

4. Creating a CNN Model using torch.nn.Sequential

`Sequential` is a container of `Modules` that can be stacked together and run at the same time. You can use `nn.Sequential` in different ways:

- Create a model directly (Section 4)
- Use it as part of the model (Section 5)

Build the network

We can create a network directly without explicitly defining a class by stacking `torch.nn.Module` layers (e.g., `torch.nn.Conv2d`, `torch.nn.ReLU`) into `Sequential`. Note that we cannot add functional layers (e.g., `torch.relu`) into a Sequential model.

First, let's define our own layer called `Flatten` since there is no `torch.nn.Module` layer to flatten an activation map. We can define our own module by inheriting `torch.nn.Module` as

shown below. Then, we need to implement only the `forward` function. Backpropagation will be performed automatically by AutoGrad.

In [0]:
```
class Flatten(torch.nn.Module):
    def forward(self, x):
        batch_size = x.shape[0]
        return x.view(batch_size, -1)
```

Now, let's create our sequential model

In [0]:
```
model2 = nn.Sequential(

    nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1),
    nn.BatchNorm2d(32),
    nn.ReLU(),

    nn.Conv2d(32, 64, 3),
    nn.BatchNorm2d(64),
    nn.ReLU(),

    Flatten(),

    nn.Linear(64*30*30, 1024),
    nn.Linear(1024, 10)
)
```

In [16]:
```
summary(model2.to(device), (3, 32, 32), device=device)
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 32, 32, 32]             896
       BatchNorm2d-2           [-1, 32, 32, 32]              64
              ReLU-3           [-1, 32, 32, 32]               0
            Conv2d-4           [-1, 64, 30, 30]          18,496
       BatchNorm2d-5           [-1, 64, 30, 30]             128
              ReLU-6           [-1, 64, 30, 30]               0
           Flatten-7                [-1, 57600]               0
            Linear-8                 [-1, 1024]      58,983,424
            Linear-9                   [-1, 10]          10,250
================================================================
Total params: 59,013,258
Trainable params: 59,013,258
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 2.52
Params size (MB): 225.12
Estimated Total Size (MB): 227.65
----------------------------------------------------------------
```

Train the model

In [17]:
```
train(model2, max_epochs=50, lr=0.001, momentum=0.9)
```

```
[Epoch  1 Iter   105/313]: train_loss = 1.8960
[Epoch  1 Iter   210/313]: train_loss = 1.6018
[Epoch  1 Iter   313/313]: train_loss = 1.4944
[Epoch  2 Iter   105/313]: train_loss = 1.1122
[Epoch  2 Iter   210/313]: train_loss = 1.1017
[Epoch  2 Iter   313/313]: train_loss = 1.1410
```

```
[Epoch  3 Iter   105/313]: train_loss = 0.8276
[Epoch  3 Iter   210/313]: train_loss = 0.8657
[Epoch  3 Iter   313/313]: train_loss = 0.8980
[Epoch  4 Iter   105/313]: train_loss = 0.6037
[Epoch  4 Iter   210/313]: train_loss = 0.6694
[Epoch  4 Iter   313/313]: train_loss = 0.7134
[Epoch  5 Iter   105/313]: train_loss = 0.4231
[Epoch  5 Iter   210/313]: train_loss = 0.4789
[Epoch  5 Iter   313/313]: train_loss = 0.5380
[Epoch  6 Iter   105/313]: train_loss = 0.3066
[Epoch  6 Iter   210/313]: train_loss = 0.3019
[Epoch  6 Iter   313/313]: train_loss = 0.3446
[Epoch  7 Iter   105/313]: train_loss = 0.1835
[Epoch  7 Iter   210/313]: train_loss = 0.1812
[Epoch  7 Iter   313/313]: train_loss = 0.2142
[Epoch  8 Iter   105/313]: train_loss = 0.1093
[Epoch  8 Iter   210/313]: train_loss = 0.0998
[Epoch  8 Iter   313/313]: train_loss = 0.1157
[Epoch  9 Iter   105/313]: train_loss = 0.0545
[Epoch  9 Iter   210/313]: train_loss = 0.0532
[Epoch  9 Iter   313/313]: train_loss = 0.0558
[Epoch 10 Iter   105/313]: train_loss = 0.0337
[Epoch 10 Iter   210/313]: train_loss = 0.0280
[Epoch 10 Iter   313/313]: train_loss = 0.0312
[Epoch 11 Iter   105/313]: train_loss = 0.0177
[Epoch 11 Iter   210/313]: train_loss = 0.0183
[Epoch 11 Iter   313/313]: train_loss = 0.0195
[Epoch 12 Iter   105/313]: train_loss = 0.0143
[Epoch 12 Iter   210/313]: train_loss = 0.0150
[Epoch 12 Iter   313/313]: train_loss = 0.0136
[Epoch 13 Iter   105/313]: train_loss = 0.0114
[Epoch 13 Iter   210/313]: train_loss = 0.0105
[Epoch 13 Iter   313/313]: train_loss = 0.0102
[Epoch 14 Iter   105/313]: train_loss = 0.0083
[Epoch 14 Iter   210/313]: train_loss = 0.0085
[Epoch 14 Iter   313/313]: train_loss = 0.0093
[Epoch 15 Iter   105/313]: train_loss = 0.0078
[Epoch 15 Iter   210/313]: train_loss = 0.0074
[Epoch 15 Iter   313/313]: train_loss = 0.0073
[Epoch 16 Iter   105/313]: train_loss = 0.0062
[Epoch 16 Iter   210/313]: train_loss = 0.0067
[Epoch 16 Iter   313/313]: train_loss = 0.0064
[Epoch 17 Iter   105/313]: train_loss = 0.0057
[Epoch 17 Iter   210/313]: train_loss = 0.0058
[Epoch 17 Iter   313/313]: train_loss = 0.0056
[Epoch 18 Iter   105/313]: train_loss = 0.0052
[Epoch 18 Iter   210/313]: train_loss = 0.0055
[Epoch 18 Iter   313/313]: train_loss = 0.0052
[Epoch 19 Iter   105/313]: train_loss = 0.0046
[Epoch 19 Iter   210/313]: train_loss = 0.0045
[Epoch 19 Iter   313/313]: train_loss = 0.0046
[Epoch 20 Iter   105/313]: train_loss = 0.0044
[Epoch 20 Iter   210/313]: train_loss = 0.0045
[Epoch 20 Iter   313/313]: train_loss = 0.0042
[Epoch 21 Iter   105/313]: train_loss = 0.0038
[Epoch 21 Iter   210/313]: train_loss = 0.0039
[Epoch 21 Iter   313/313]: train_loss = 0.0041
[Epoch 22 Iter   105/313]: train_loss = 0.0035
[Epoch 22 Iter   210/313]: train_loss = 0.0037
[Epoch 22 Iter   313/313]: train_loss = 0.0037
[Epoch 23 Iter   105/313]: train_loss = 0.0034
[Epoch 23 Iter   210/313]: train_loss = 0.0034
[Epoch 23 Iter   313/313]: train_loss = 0.0033
[Epoch 24 Iter   105/313]: train_loss = 0.0031
[Epoch 24 Iter   210/313]: train_loss = 0.0032
[Epoch 24 Iter   313/313]: train_loss = 0.0033
[Epoch 25 Iter   105/313]: train_loss = 0.0031
```

Evaluate the model on the test set

In [18]:
```
evaluate(model2)
```

```
Accuracy = 56.45%
```

---

5. Creating Block Layers

Build the Network

We can also use `nn.Sequential` to group similar layers in a block. For example, refer to the AlexNet architecture in Section 2. We can group the layer 1 and 2 into a block (`conv_block_1`) and layer 3 and 4 into another block (`conv_block_2`). The two linear layer, layer 5 and 6, can be grouped into another block (`decoder`).

Block	Layer	Name	Description	Input Shape	OutputShape
conv_block_1	1	conv2d-1	Conv2d, f=3, c=32, s=1, p=1	(?, 3, 32, 32)	(?, 32, 32, 32)
	2	BatchNorm2d-2	BatchNorm2d, c=32	(?, 32, 32, 32)	(?, 32, 32, 32)
	-	Relu	relu	(?, 32, 32, 32)	(?, 32, 32, 32)
conv_block_2	3	conv2d-3	Conv2d, f=3, c=64, s=1, p=0	(?, 32, 32, 32)	(?, 64, 30, 30)
	4	BatchNorm2d-4	BatchNorm2d, c=64	(?, 64, 30, 30)	(?, 64, 30, 30)
	-	Relu	relu	(?, 64, 30, 30)	(?, 64, 30, 30)
-	-	-	flatten	(?, 64, 30, 30)	(?, 57600)
decoder	5	Linear-5	Linear, c =1024	(?, 57600)	(?, 1024)
	6	Linear-6	Linear, c=10	(?, 1024)	(?, 10)

The following code shows how to do this.

In [0]:
```
class Net3(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv_block1 = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)   ,
            nn.BatchNorm2d(32),
            nn.ReLU()
        )

        self.conv_block2 = nn.Sequential(
            nn.Conv2d(32, 64, 3),
            nn.BatchNorm2d(64),
            nn.ReLU()
        )

        self.decoder = nn.Sequential(
            nn.Linear(64*30*30, 1024),
            nn.Linear(1024, 10)
        )

    def forward(self, x):
```

```
        x = self.conv_block1(x)
        x = self.conv_block2(x)

        x = x.view(x.size(0), -1) # flatten

        x = self.decoder(x)

        return x
```

Let's see what we have built

In [20]:

```
model3 = Net3()
summary(model3.to(device), (3, 32, 32), device=device)
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 32, 32, 32]             896
       BatchNorm2d-2           [-1, 32, 32, 32]              64
              ReLU-3           [-1, 32, 32, 32]               0
            Conv2d-4           [-1, 64, 30, 30]          18,496
       BatchNorm2d-5           [-1, 64, 30, 30]             128
              ReLU-6           [-1, 64, 30, 30]               0
            Linear-7                 [-1, 1024]      58,983,424
            Linear-8                   [-1, 10]          10,250
================================================================
Total params: 59,013,258
Trainable params: 59,013,258
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.01
Forward/backward pass size (MB): 2.08
Params size (MB): 225.12
Estimated Total Size (MB): 227.21
----------------------------------------------------------------
```

Train the model

In [21]:

```
train(model3, max_epochs=50, lr=0.001, momentum=0.9)
```

```
[Epoch  1 Iter   105/313]: train_loss = 1.8814
[Epoch  1 Iter   210/313]: train_loss = 1.5424
[Epoch  1 Iter   313/313]: train_loss = 1.4594
[Epoch  2 Iter   105/313]: train_loss = 1.0586
[Epoch  2 Iter   210/313]: train_loss = 1.0667
[Epoch  2 Iter   313/313]: train_loss = 1.0771
[Epoch  3 Iter   105/313]: train_loss = 0.7783
[Epoch  3 Iter   210/313]: train_loss = 0.8077
[Epoch  3 Iter   313/313]: train_loss = 0.8257
[Epoch  4 Iter   105/313]: train_loss = 0.5551
[Epoch  4 Iter   210/313]: train_loss = 0.6273
[Epoch  4 Iter   313/313]: train_loss = 0.6278
[Epoch  5 Iter   105/313]: train_loss = 0.3824
[Epoch  5 Iter   210/313]: train_loss = 0.4342
[Epoch  5 Iter   313/313]: train_loss = 0.4675
[Epoch  6 Iter   105/313]: train_loss = 0.2445
[Epoch  6 Iter   210/313]: train_loss = 0.2674
[Epoch  6 Iter   313/313]: train_loss = 0.2835
[Epoch  7 Iter   105/313]: train_loss = 0.1639
[Epoch  7 Iter   210/313]: train_loss = 0.1542
[Epoch  7 Iter   313/313]: train_loss = 0.1753
[Epoch  8 Iter   105/313]: train_loss = 0.0869
[Epoch  8 Iter   210/313]: train_loss = 0.0931
[Epoch  8 Iter   313/313]: train_loss = 0.1007
[Epoch  9 Iter   105/313]: train_loss = 0.0538
```

```
[Epoch  9 Iter   210/313]: train_loss = 0.0441
[Epoch  9 Iter   313/313]: train_loss = 0.0412
[Epoch 10 Iter   105/313]: train_loss = 0.0248
[Epoch 10 Iter   210/313]: train_loss = 0.0233
[Epoch 10 Iter   313/313]: train_loss = 0.0262
[Epoch 11 Iter   105/313]: train_loss = 0.0169
[Epoch 11 Iter   210/313]: train_loss = 0.0154
[Epoch 11 Iter   313/313]: train_loss = 0.0167
[Epoch 12 Iter   105/313]: train_loss = 0.0130
[Epoch 12 Iter   210/313]: train_loss = 0.0117
[Epoch 12 Iter   313/313]: train_loss = 0.0117
[Epoch 13 Iter   105/313]: train_loss = 0.0089
[Epoch 13 Iter   210/313]: train_loss = 0.0099
[Epoch 13 Iter   313/313]: train_loss = 0.0094
[Epoch 14 Iter   105/313]: train_loss = 0.0079
[Epoch 14 Iter   210/313]: train_loss = 0.0081
[Epoch 14 Iter   313/313]: train_loss = 0.0078
[Epoch 15 Iter   105/313]: train_loss = 0.0068
[Epoch 15 Iter   210/313]: train_loss = 0.0065
[Epoch 15 Iter   313/313]: train_loss = 0.0070
[Epoch 16 Iter   105/313]: train_loss = 0.0059
[Epoch 16 Iter   210/313]: train_loss = 0.0059
[Epoch 16 Iter   313/313]: train_loss = 0.0059
[Epoch 17 Iter   105/313]: train_loss = 0.0055
[Epoch 17 Iter   210/313]: train_loss = 0.0051
[Epoch 17 Iter   313/313]: train_loss = 0.0053
[Epoch 18 Iter   105/313]: train_loss = 0.0044
[Epoch 18 Iter   210/313]: train_loss = 0.0052
[Epoch 18 Iter   313/313]: train_loss = 0.0049
[Epoch 19 Iter   105/313]: train_loss = 0.0046
```

Evaluate the model

In [22]:

```
evaluate(model3)
```

```
Accuracy = 58.20%
```

In [0]:

```

```